Complete FastAPI Notes – Beginner to Advanced

Complete Learning Notes
Python Backend Development

Ye notes is tarah arrange kiye hain ke tum FastAPI ko zero se advanced level tak samajh sako aur apne projects mein independently backend develop kar sako.

Hum har concept ko 3 cheezon ke through samjhenge:

1. Definition: Concept kya hai?
2. Example: Code mein kaise use hota hai?
3. Practical understanding: Real-world project mein iska kya use hai?

Examples ke liye hum ek Student Management System banane ka scenario use karenge.

PART 1 – Introduction to FastAPI

1. What is FastAPI?

FastAPI is a modern, high-performance Python web framework used to build APIs and backend applications.

Simple words mein:

FastAPI Python ka framework hai jo frontend aur database ke darmiyan communication establish karta hai.

For example, agar tum Next.js frontend bana rahi ho aur Python backend use karna chahti ho, to FastAPI dono ko connect kar sakta hai.

APIs Without Tears: A Gentle Intro to FastAPI | by chaithanya k | Medium
medium.com

Why do we use FastAPI?

- High performance
- Easy to learn
- Automatic API documentation

- Data validation
- Type hints support
- Async programming support
- Easy integration with databases
- Authentication and authorization support

2. What is an API?

API stands for Application Programming Interface.

An API is a communication interface that allows two software applications to exchange data.

Example:

Suppose you have:

- Frontend: Next.js
- Backend: FastAPI
- Database: PostgreSQL

When a student opens their dashboard:

1. Frontend sends a request to FastAPI.
2. FastAPI processes the request.
3. FastAPI fetches student information from the database.
4. FastAPI returns the data to the frontend.
5. Frontend displays the student information.

3. What is a Framework?

A framework is a collection of pre-built tools, libraries, and rules that help developers build applications efficiently.

Examples:

Framework	Language	Purpose
FastAPI	Python	API development

Django	Python	Full-stack web development
Flask	Python	Web applications and APIs
Express.js	JavaScript	Backend development
Next.js	JavaScript/TypeScript	Full-stack web development

4. FastAPI vs Flask vs Django

Feature	FastAPI	Flask	Django
Type	API framework	Micro framework	Full-stack framework
Performance	High	Depends on implementation	Depends on implementation
Validation	Built-in	Manual/extensions	Built-in forms/serializers via DRF
Documentation	Automatic	Extensions	Additional tools
Async	Native support	Supported	Supported
Learning	Easy	Easy	Comparatively complex

Remember: FastAPI is particularly useful when building REST APIs, AI/ML backends, and modern applications.

PART 2 – Installation and Project Setup

5. Installing FastAPI

First, create a project folder.

```
None
mkdir FastAPI-Learning
cd FastAPI-Learning
```

Create a virtual environment:

```
None
python -m venv venv
```

Activate it:

```
None
venv\Scripts\activate
```

Install FastAPI and Uvicorn:

```
None
pip install "fastapi[standard]"
```

Check installation:

```
None
pip show fastapi
```

6. What is a Virtual Environment?

A virtual environment is an isolated Python environment used to manage dependencies for a particular project.

Example:

Project A requires FastAPI version X, while Project B requires another version.

Virtual environments allow both projects to maintain separate dependencies.

7. What is Uvicorn?

Uvicorn is an ASGI server used to run FastAPI applications.

ASGI stands for Asynchronous Server Gateway Interface.

It handles communication between the client and your Python application.

Run FastAPI

Suppose your main file is `main.py`.

```
None
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Welcome to FastAPI"}
```

Run:

```
None
uvicorn main:app --reload
```

Explanation:

Command	Meaning

uvicorn	ASGI server
main	Python filename
app	FastAPI application object
--reload	Automatically restarts server when code changes

Open these URLs:

- API: `http://127.0.0.1:8000`
- Swagger UI: `http://127.0.0.1:8000/docs`
- ReDoc: `http://127.0.0.1:8000/redoc`

Dependencies - FastAPI
Path Parameters - FastAPI
FastAPI Path Operations for Teams - Other Models - SQLModel
6

8. What is Swagger UI?

Swagger UI is an interactive API documentation interface.

It allows developers to:

- View all API endpoints.
- Understand request parameters.
- Send API requests.
- Test POST, GET, PUT and DELETE operations.
- View API responses.

Important: You can test your backend without building a frontend using Swagger UI.

PART 3 – FastAPI Fundamentals

9. What is an Endpoint?

An endpoint is a specific URL through which an API provides a particular functionality.

Example:

```
None
@app.get("/students")
def get_students():
    return {"students": ["Ali", "Sara", "Ahmed"]}
```

Here:

- `/students` = Endpoint path
- `GET` = HTTP method
- `get_students()` = Function
- Returned dictionary = API response

10. HTTP Methods

HTTP methods define what action a client wants to perform on a server.

These are extremely important in backend development.

Method	Purpose	Example
GET	Retrieve data	Get all students
POST	Create new data	Add a student
PUT	Replace/update data	Update student

PATCH	Partially update data	Update student email
DELETE	Delete data	Delete student

GET Example

```
None
@app.get("/students")
def get_students():
    return {"message": "All students"}
```

POST Example

```
None
@app.post("/students")
def create_student():
    return {"message": "Student created"}
```

PUT Example

```
None
@app.put("/students/{student_id}")
def update_student(student_id: int):
    return {"message": f"Student {student_id} updated"}
```

DELETE Example

```
None
@app.delete("/students/{student_id}")
def delete_student(student_id: int):
    return {"message": f"Student {student_id} deleted"}
```

11. Path Parameters

Path parameters are variables included directly in the URL.

Example:

```
None
@app.get("/students/{student_id}")
def get_student(student_id: int):
    return {"student_id": student_id}
```

Request:

```
None
http://127.0.0.1:8000/students/10
```

Response:

```
None
{
    "student_id": 10
}
```

Here, `10` is the path parameter.

Use: When you want to retrieve a particular student, product, user, or notice.

12. Query Parameters

Query parameters are additional values passed through the URL after `?`.

Example:

```
None
@app.get("/students")
def get_students(department: str, semester: int):
    return {
        "department": department,
        "semester": semester
    }
```

Request:

```
None
/students?department=CS&semester=4
```

Response:

```
None
{
    "department": "CS",
    "semester": 4
}
```

Difference between Path and Query Parameters

Path Parameter	Query Parameter
Part of URL path	Comes after `?`
Usually identifies a resource	Usually filters or modifies results
`/students/10`	`/students?semester=4`

13. Optional Parameters

Sometimes a parameter is not required.

```
None
from typing import Optional

@app.get("/students")
def get_students(semester: Optional[int] = None):
    return {"semester": semester}
```

Now users can call:

```
None
/students
```

Or:

```
None
/students?semester=4
```

14. Default Parameters

You can provide default values.

```
None
@app.get("/products")
def get_products(limit: int = 10):
    return {"limit": limit}
```

If no limit is provided, FastAPI uses 10.

PART 4 – Request Body and Pydantic

15. What is a Request Body?

A request body is the data sent by the client to the server, usually when creating or updating a resource.

For example, when a user registers, frontend sends:

```
None
{
    "name": "Sara",
    "email": "sara@gmail.com",
    "age": 21
}
```

FastAPI receives this data through a request body.

16. What is Pydantic?

Pydantic is a Python library used for data validation, parsing, and serialization.

In simple words:

Pydantic checks whether incoming data has the correct types and structure.

Example:

```
None
from pydantic import BaseModel

class Student(BaseModel):
    name: str
    email: str
    age: int
```

Now use this model in FastAPI:

```
None
@app.post("/students")
def create_student(student: Student):
    return {
        "message": "Student created",
        "student": student
    }
```

Example request:

```
None
{
    "name": "Sara",
    "email": "sara@gmail.com",
    "age": 21
}
```

If the age is provided as an invalid value, Pydantic validates the input and FastAPI returns a validation error.

Important Pydantic Concepts

Concept	Meaning
BaseModel	Base class for creating schemas
Field	Adds validation and metadata
Optional	Makes a field optional
model_dump()	Converts model into dictionary
model_validate()	Validates data against a model
EmailStr	Validates email format

Example with Field:

```
None
from pydantic import BaseModel, Field, EmailStr

class Student(BaseModel):
    name: str = Field(min_length=3, max_length=50)
    email: EmailStr
    age: int = Field(gt=0, lt=100)
```

Here:

- `min_length=3`: Name must have at least 3 characters.
- `max_length=50`: Name cannot exceed 50 characters.
- `EmailStr`: Email must have a valid email format.
- `gt=0`: Age must be greater than zero.
- `lt=100`: Age must be less than 100.

Install email validation support:

```
None
pip install email-validator
```

17. Request Body vs Path vs Query

Type	Example	Purpose
Path	`/students/5`	Identify student
Query	`/students?semester=4`	Filter students
Body	JSON data	Send new student information

This distinction is very important for API development.

PART 5 – Response Handling

18. What is a Response?

A response is the data returned by the server after processing a client request.

Example:

```
None
@app.get("/")
def home():
    return {
        "message": "API is working",
        "status": "success"
    }
```

FastAPI automatically converts Python dictionaries and lists into JSON responses.

19. What is JSON?

JSON stands for JavaScript Object Notation.

It is a lightweight data format used for exchanging information between applications.

Example:

```
None
{
    "name": "Sara",
    "department": "Computer Science",
    "semester": 4
}
```

JSON is widely used in frontend-backend communication.

20. Response Model

A response model defines the structure of data that an API should return.

Example:

```
None
from pydantic import BaseModel

class StudentResponse(BaseModel):
    name: str
```

```
    email: str

@app.get("/student", response_model=StudentResponse)
def get_student():
    return {
        "name": "Sara",
        "email": "sara@gmail.com",
        "password": "secret123"
    }
```

The response model filters out fields not included in the schema.

Actual response:

```
None
{
    "name": "Sara",
    "email": "sara@gmail.com"
}
```

Why is it important?

It helps protect sensitive fields, maintain consistent API responses, and validate output data.

21. HTTP Status Codes

Status codes tell the client whether a request succeeded or failed.

Code	Meaning	Example
200	OK	Data retrieved
201	Created	New student created

204	No Content	Successfully deleted
400	Bad Request	Invalid request
401	Unauthorized	Authentication required
403	Forbidden	Access denied
404	Not Found	Student not found
422	Validation Error	Invalid input data
500	Internal Server Error	Server-side issue

Example:

```
None
from fastapi import status

@app.post(
    "/students",
    status_code=status.HTTP_201_CREATED
)
def create_student():
    return {"message": "Student created"}
```

22. HTTPException

HTTPException is used to return HTTP errors from your API.

Example:

```
None
from fastapi import HTTPException

@app.get("/students/{student_id}")
def get_student(student_id: int):

    if student_id != 1:
        raise HTTPException(
            status_code=404,
            detail="Student not found"
        )

    return {"name": "Sara"}
```

Remember: Use `raise HTTPException`, not `return HTTPException`.

PART 6 – CRUD Operations

23. What is CRUD?

CRUD stands for:

- Create
- Read
- Update
- Delete

These are the four fundamental operations of database applications.

[What are the steps to test CRUD operations for APIs?
linkedin.com]

Operation	HTTP Method	Example
Create	POST	Add student

Read	GET	View students
Update	PUT/PATCH	Edit student
Delete	DELETE	Remove student

Complete CRUD Example Using a Python List

```
None
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class Student(BaseModel):
    name: str
    age: int
    department: str

students = []

# CREATE
@app.post("/students")
def create_student(student: Student):

    students.append(student.model_dump())

    return {"message": "Student added"}

# READ ALL
@app.get("/students")
def get_students():

    return students
```

```
# READ ONE
@app.get("/students/{student_id}")
def get_student(student_id: int):

    if student_id < 0 or student_id >= len(students):
        raise HTTPException(404, "Student not found")

    return students[student_id]

# UPDATE
@app.put("/students/{student_id}")
def update_student(student_id: int, student: Student):

    if student_id < 0 or student_id >= len(students):
        raise HTTPException(404, "Student not found")

    students[student_id] = student.model_dump()

    return {"message": "Student updated"}

# DELETE
@app.delete("/students/{student_id}")
def delete_student(student_id: int):

    if student_id < 0 or student_id >= len(students):
        raise HTTPException(404, "Student not found")

    students.pop(student_id)

    return {"message": "Student deleted"}
```

Important
This is an educational example. A Python list stores data temporarily, so data disappears when the application restarts. Real applications should use a database.

PART 7 – Database Integration

24. What is a Database?

A database is an organized collection of data that can be stored, retrieved, updated, and deleted.

Examples:

- MySQL
- PostgreSQL
- SQLite
- MongoDB
- Oracle

25. What is ORM?

ORM stands for Object Relational Mapping.

ORM allows developers to interact with relational databases using Python classes and objects instead of writing SQL queries for every operation.

Example:

Instead of writing:

```
None
SELECT * FROM students;
```

You can use Python ORM methods to retrieve student records.

26. What is SQLAlchemy?

SQLAlchemy is a Python SQL toolkit and ORM used to connect Python applications with relational databases.

It allows us to:

- Create database models.
- Establish database connections.
- Insert records.
- Retrieve records.
- Update records.
- Delete records.

Install:

```
None
pip install sqlalchemy
```

27. Database Connection

Example using SQLite:

```
None
from sqlalchemy import create_engine

DATABASE_URL = "sqlite:///./students.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)
```

Explanation:

- `create_engine()` creates the database engine.
- `DATABASE_URL` specifies the database location.
- `students.db` is the SQLite database file.

For PostgreSQL, you can use a URL such as:

```
None
postgresql+psycopg://username:password@localhost:5432/mydatabase
```

Install its driver:

```
None
pip install "psycopg[binary]"
```

28. Database Models

A database model represents a table in the database.

Example:

```
None
from sqlalchemy.orm import DeclarativeBase
from sqlalchemy import Column, Integer, String

class Base(DeclarativeBase):
    pass

class Student(Base):
    __tablename__ = "students"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100))
    email = Column(String(100), unique=True)
    age = Column(Integer)
```

Here:

- `__tablename__`: Database table name.
- `id`: Primary key.
- `primary_key=True`: Uniquely identifies each record.
- `unique=True`: Prevents duplicate emails.
- `Column`: Defines a table column.

29. What is a Primary Key?

A primary key is a column that uniquely identifies each record in a database table.

Example:

id	name	email
1	Sara	sara@gmail.com

2	Ali	ali@gmail.com
3	Ahmed	ahmed@gmail.com

Here, `id` is the primary key.

30. What is a Foreign Key?

A foreign key is a column that establishes a relationship between two database tables.

Example:

Students Table

id	name
1	Sara
2	Ali

Courses Table

id	course_name	student_id
1	AI	1
2	DAA	1

3	Database	2

Here, `student_id` in Courses references the `id` of Students.

31. What is a Database Session?

A database session is an interface used to communicate with the database during a unit of work.

It allows you to:

- Execute queries.
- Add records.
- Update records.
- Delete records.
- Commit transactions.
- Roll back failed transactions.

Example:

```
None
from sqlalchemy.orm import sessionmaker

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    autocommit=False
)
```

A common FastAPI database dependency:

```
None
from fastapi import Depends

def get_db():
    db = SessionLocal()

    try:
```

```
        yield db
    finally:
        db.close()
```

Use it in an endpoint:

```
None
@app.get("/students")
def get_students(db = Depends(get_db)):
    return db.query(Student).all()
```

32. What is Alembic?

Alembic is a database migration tool used with SQLAlchemy.

It helps manage changes in database structure over time.

For example:

- Adding a new column.
- Removing a column.
- Creating a new table.
- Changing a column type.

Install:

```
None
pip install alembic
```

Initialize:

```
None
alembic init migrations
```

Think of migrations as version control for your database structure.

PART 8 – Dependency Injection

33. What is Dependency Injection?

Dependency Injection (DI) is a technique in which a function receives the resources or services it needs from an external source rather than creating them itself.

FastAPI provides this through `Depends()`.

Example:

```
None
from fastapi import Depends

def common_parameters():
    return {
        "page": 1,
        "limit": 10
    }

@app.get("/students")
def get_students(data = Depends(common_parameters)):
    return data
```

Here, FastAPI automatically calls `common_parameters()` and passes its result to the endpoint.

Real-world uses

- Database sessions.
- Authentication.
- Authorization.
- Pagination.
- Reusable services.
- Configuration management.

Remember: `Depends()` is one of the most important concepts in professional FastAPI applications.

PART 9 – Project Structure and APIRouter

34. What is APIRouter?

APIRouter is used to organize endpoints into separate modules.

Suppose your project has:

- Authentication
- Students
- Teachers
- Courses
- Notices
- Complaints

Writing everything inside `main.py` makes the code difficult to maintain.

APIRouter solves this problem.

Recommended Project Structure

FastAPI-Project/

main.py

app/

database.py

models.py

schemas.py

dependencies.py

routers/

auth.py

students.py

courses.py

services/

crud/

requirements.txt

.env

Router Example

In `routers/students.py`:

```
None
from fastapi import APIRouter

router = APIRouter(
    prefix="/students",
    tags=["Students"]
)

@router.get("/")
def get_students():
    return {"message": "All students"}

@router.post("/")
def create_student():
    return {"message": "Student created"}
```

In `main.py`:

```
None
from fastapi import FastAPI
from routers.students import router as student_router

app = FastAPI()

app.include_router(student_router)
```

Now your endpoints are:

```
None
GET  /students/
POST /students/
```

What is Prefix?

Prefix allows you to add a common path to all routes inside a router.

What are Tags?

Tags group endpoints in Swagger documentation.

PART 10 – Authentication and Authorization

35. What is Authentication?

Authentication is the process of verifying the identity of a user.

Example:

A student enters their email and password to log in.

The backend verifies whether the credentials are correct.

36. What is Authorization?

Authorization determines what an authenticated user is allowed to access.

Example:

- Admin can manage all students.
- Teacher can manage their courses.
- Student can view their own profile.

Authentication vs Authorization

Authentication	Authorization
Who are you?	What can you access?
Verifies identity	Checks permissions
Login	Role-based access

37. What is JWT?

JWT stands for JSON Web Token.

It is a token format commonly used for secure authentication between frontend and backend.

[How JWT Works: Step-by-Step Guide for Beginners | ADevGuide](adevguide.com)

Typical JWT authentication flow:

1. User enters login credentials.
2. Frontend sends credentials to backend.
3. Backend verifies credentials.
4. Backend generates a signed JWT.
5. Frontend receives the token.
6. Frontend sends the token with protected requests.
7. Backend verifies the token before granting access.

JWT has three parts:

```
Header.Payload.Signature
```

Part	Purpose

Header	Token type and signing algorithm
Payload	Claims such as user ID and expiry
Signature	Verifies token integrity

Important: JWT payload is encoded, not encrypted. Never store passwords or secrets inside it.

38. Password Hashing

Passwords should never be stored as plain text.

Hashing converts a password into a one-way representation.

Example:

```
None
Original: sara123
Hashed: $2b$12$...
```

A common Python library is `pwdlib`.

```
None
pip install "pwdlib[argon2]"
```

Argon2 is a password hashing algorithm.

39. OAuth2

OAuth2 is an authorization framework used to grant limited access to protected resources.

FastAPI provides OAuth2 security utilities.

Important concepts:

- OAuth2PasswordBearer
- OAuth2PasswordRequestForm
- Bearer Token
- Access Token
- Token Expiration

Important distinction: OAuth2 is an authorization framework; a JWT is a token format. They are related but not the same thing.

PART 11 – Middleware and CORS

40. What is Middleware?

Middleware is code that executes during the request-response processing cycle.

It can process requests before they reach an endpoint and responses before they are returned to the client.

Example uses:

- Logging
- Request timing
- Security headers
- Authentication checks
- Monitoring

Example:

```
None
from fastapi import Request
import time

@app.middleware("http")
async def process_request(request: Request, call_next):

    start = time.time()

    response = await call_next(request)

    process_time = time.time() - start
```

```
    response.headers["X-Process-Time"] = str(process_time)

    return response
```

41. What is CORS?

CORS stands for Cross-Origin Resource Sharing.

It is a browser security mechanism that controls whether a frontend from one origin can access resources from another origin.

Example:

Frontend:

```
None
http://localhost:3000
```

Backend:

```
None
http://localhost:8000
```

These have different origins because their ports are different.

Configure CORS:

```
None
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
```

```
)
```

Important:

- `allow_origins`: Which frontend origins are allowed.
- `allow_methods`: Which HTTP methods are allowed.
- `allow_headers`: Which request headers are allowed.
- `allow_credentials`: Whether credentials can be included.

For production, specify trusted origins instead of allowing every origin.

PART 12 – Async and Await

42. What is Asynchronous Programming?

Asynchronous programming allows a program to work efficiently while waiting for operations such as database queries, API requests, or file operations.

Example:

A backend calls an external AI API. Instead of blocking the current execution while waiting, it can use asynchronous operations.

43. Difference between Sync and Async

Synchronous	Asynchronous
Uses `def`	Often uses `async def`
Blocking operations can block execution	Supports non-blocking I/O
Executes sequentially while waiting	Can handle other tasks while waiting

Suitable for ordinary synchronous libraries	Useful for async-compatible libraries

Synchronous Example

```
None
@app.get("/sync")
def sync_function():
    return {"message": "Synchronous"}
```

Asynchronous Example

```
None
@app.get("/async")
async def async_function():
    return {"message": "Asynchronous"}
```

What is Await?

`await` pauses the current coroutine until an asynchronous operation completes, while allowing the event loop to run other tasks.

Example:

```
None
import asyncio

@app.get("/wait")
async def wait_function():

    await asyncio.sleep(2)

    return {"message": "Completed"}
```

Important: `async def` does not automatically make blocking code non-blocking. Use async-compatible libraries for I/O operations or appropriate thread handling for blocking operations.

PART 13 – Background Tasks and File Handling

44. What are Background Tasks?

Background tasks allow certain operations to execute after a response has been sent.

Examples:

- Sending emails.
- Generating reports.
- Logging activity.
- Sending notifications.

Example:

```
None
from fastapi import BackgroundTasks

def send_email(email: str):
    print(f"Email sent to {email}")

@app.post("/register")
def register(background_tasks: BackgroundTasks):

    background_tasks.add_task(
        send_email,
        "sara@gmail.com"
    )

    return {"message": "Registration successful"}
```

For large or long-running tasks, use a dedicated task queue such as Celery rather than relying on in-process background tasks.

45. File Upload

FastAPI supports file uploads through `UploadFile`.

Install:

```
None
pip install python-multipart
```

Example:

```
None
from fastapi import UploadFile, File

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):

    return {
        "filename": file.filename,
        "content_type": file.content_type
    }
```

`UploadFile` is useful for images, PDFs, documents, and other files.

46. Form Data

Form data is commonly used when frontend submits HTML forms.

Example:

```
None
from fastapi import Form

@app.post("/login")
def login(
    username: str = Form(...),
    password: str = Form(...)
):
    return {"username": username}
```

Difference:

- JSON Body: Common for API data exchange.
- Form Data: Common for form submissions and file uploads.

PART 14 – Advanced FastAPI Concepts

47. Pagination

Pagination means dividing a large amount of data into smaller pages.

Example: University has 5,000 students. Instead of returning all students at once, return 10 students per page.

```
None
@app.get("/students")
def get_students(
    page: int = 1,
    limit: int = 10
):
    return {
        "page": page,
        "limit": limit
    }
```

Request:

```
None
/students?page=2&limit=10
```

Common pagination parameters:

- `page`: Current page number.
- `limit`: Number of records per page.
- `skip`: Number of records to skip.

Formula:

$$skip = (page - 1) \times limit$$

48. Filtering and Searching

Filtering allows users to retrieve specific records based on conditions.

Example:

```
None
@app.get("/students")
def get_students(
    department: str = None,
    semester: int = None
):
    return {
        "department": department,
        "semester": semester
    }
```

Real-world applications:

- Search students by name.
- Filter notices by category.
- Filter complaints by status.
- Search products by price.

49. Environment Variables

Environment variables store configuration values outside the source code.

Examples:

- Database URL
- Secret keys
- API keys
- Application settings

Create `.env`:

```
None
DATABASE_URL=postgresql+psycopg://user:password@localhost/db
SECRET_KEY=your-secret-key
```

```
GROQ_API_KEY=your-api-key
```

Install:

```
None
pip install pydantic-settings
```

Example:

```
None
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):

    DATABASE_URL: str
    SECRET_KEY: str

    model_config = SettingsConfigDict(
        env_file=".env"
    )

settings = Settings()
```

Now:

```
None
print(settings.DATABASE_URL)
```

Never commit real `.env` secrets to GitHub.

50. API Versioning

API versioning allows developers to maintain multiple versions of an API.

Example:

```
None
/api/v1/students
/api/v2/students
```

Why is it useful?

Suppose you change your API response structure. Existing frontend applications might break.

Versioning allows old and new clients to use different API versions.

51. Testing APIs

Testing ensures that your endpoints work correctly.

FastAPI provides `TestClient`.

Install:

```
None
pip install pytest httpx
```

Example:

```
None
from fastapi.testclient import TestClient

client = TestClient(app)

def test_home():

    response = client.get("/")

    assert response.status_code == 200
    assert response.json() == {
        "message": "Welcome to FastAPI"
    }
```

Run tests:

```
None
pytest
```

Important testing concepts:

- Unit Testing
- Integration Testing
- API Testing
- TestClient
- Pytest
- Mocking

52. WebSockets

WebSockets provide two-way communication between the client and server over a persistent connection.

Unlike ordinary HTTP requests, WebSockets allow the server to send updates to the client without waiting for another request.

Tired of Slow Loading Pages? Speed Up Your Web App with WebSockets - DEV Community
dev.to

Real-world examples:

- Live chat applications.
- Real-time notifications.
- Live dashboards.
- Collaborative applications.
- Real-time complaint status updates.

Basic example:

```
None
from fastapi import WebSocket

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):

    await websocket.accept()
```

```
    while True:
        data = await websocket.receive_text()

        await websocket.send_text(
            f"Message received: {data}"
        )
```

53. Deployment

Deployment means making your backend application accessible on a server or cloud platform.

Common deployment platforms:

- Railway
- Render
- AWS
- Azure
- Google Cloud

Typical deployment workflow:

1. Develop the FastAPI backend.
2. Configure environment variables.
3. Connect the database.
4. Create `requirements.txt`.
5. Configure the production server.
6. Deploy the backend.
7. Connect the frontend to the deployed API.

Generate requirements:

```
None
pip freeze > requirements.txt
```

Run locally:

```
None
uvicorn main:app --reload
```

Production-style command:

```
None
uvicorn main:app --host 0.0.0.0 --port 8000
```

For production, also configure HTTPS, logging, database migrations, security, and process management.

PART 15 – FastAPI with Frontend

54. How does FastAPI communicate with React or Next.js?

Suppose you have:

- Frontend: Next.js
- Backend: FastAPI
- Database: PostgreSQL

[Crear una aplicación web full stack con nextjs y python fastapi o django
es.fiverr.com]

Frontend sends a request:

```
None
const response = await fetch(
    "http://localhost:8000/students"
);

const data = await response.json();

console.log(data);
```

FastAPI receives it:

```
None
@app.get("/students")
```

```
def get_students():

    return [
        {"id": 1, "name": "Sara"},
        {"id": 2, "name": "Ali"}
    ]
```

Frontend receives JSON and displays the information.

Complete Communication Flow

1. User clicks a button on the frontend.
2. React/Next.js sends an HTTP request.
3. FastAPI receives the request.
4. FastAPI validates the input.
5. FastAPI processes business logic.
6. FastAPI communicates with the database if required.
7. FastAPI returns JSON.
8. Frontend displays the response.

55. What is REST API?

REST stands for Representational State Transfer.

REST is an architectural style for designing network APIs.

Common REST principles:

- Client-server architecture.
- Stateless communication.
- Resource-based URLs.
- Standard HTTP methods.
- Consistent representations.

Example:

Endpoint	Operation

`/api/students`	Get all students
`/api/students/1`	Get student 1
`/api/students` (POST)	Create student
`/api/students/1` (PUT)	Update student 1
`/api/students/1` (DELETE)	Delete student 1

PART 16 – FastAPI for AI/ML Applications

Since FastAPI is also widely used for AI/ML backend development, these concepts are particularly useful.

56. FastAPI as an ML Model Backend

Suppose you trained a machine learning model using Scikit-learn.

You want to create an API that accepts input and returns predictions.

Architecture:

Simple Machine Learning Model Deployment using FastAPI | by Faisal Malik Widya Prasetya | Medium
faisalmalikwp.medium.com

Example:

```
None
from fastapi import FastAPI
```

```
from pydantic import BaseModel
import joblib

app = FastAPI()

model = joblib.load("model.pkl")

class PredictionInput(BaseModel):
    age: int
    income: float

@app.post("/predict")
def predict(data: PredictionInput):

    prediction = model.predict([[
        data.age,
        data.income
    ]])

    return {
        "prediction": prediction.tolist()
    }
```

Here:

- `model.pkl`: Trained model file.
- `PredictionInput`: Input validation schema.
- `/predict`: Prediction endpoint.
- `model.predict()`: Generates prediction.
- `tolist()`: Converts NumPy output into a JSON-compatible list.

57. FastAPI with LLMs

FastAPI can also be used to develop AI-powered applications.

For example:

- AI Chatbots

- RAG Applications
- AI Agents
- Document Question Answering
- Text Summarization
- AI Automation

Example architecture:

Build an AI Assistant for Your CRM | by Kintone | Medium
kintone.medium.com

Typical workflow:

1. Frontend sends a question.
2. FastAPI receives the question.
3. Backend retrieves relevant documents.
4. Retrieved context is sent to an LLM.
5. LLM generates an answer.
6. FastAPI returns the response.

PART 17 – Important FastAPI Terminology

Term	Definition
API	Interface for software communication
Endpoint	Specific API URL
Route	URL path connected to a function
HTTP	Protocol for web communication
REST	API architectural style

JSON	Data exchange format
Uvicorn	ASGI server
ASGI	Asynchronous Server Gateway Interface
Pydantic	Data validation library
Schema	Structure of input/output data
ORM	Object Relational Mapping
SQLAlchemy	Python SQL toolkit and ORM
CRUD	Create, Read, Update, Delete
JWT	JSON Web Token
OAuth2	Authorization framework
Middleware	Request/response processing layer
CORS	Cross-Origin Resource Sharing

Dependency Injection	Providing reusable dependencies
WebSocket	Two-way persistent communication
Alembic	Database migration tool
Async	Asynchronous programming
Pagination	Dividing results into pages

PART 18 – Common FastAPI Mistakes

Mistake	Correct Approach
Forgetting to activate venv	Activate before running
Wrong module path	Use correct `filename:app`
Using GET to create records	Use POST
Storing passwords directly	Use secure password hashing
Writing all routes in main.py	Use APIRouter

No input validation	Use Pydantic
Hardcoding API keys	Use environment variables
Not handling errors	Use HTTPException
Forgetting CORS	Configure allowed frontend origins
Using a Python list as permanent storage	Use a database
Returning sensitive information	Use response models
Using blocking code in async endpoints	Use async-compatible operations

PART 19 – Important Questions for Interviews and Exams

In concepts ko apne words mein explain karna seekho.

1. What is FastAPI?
2. Why is FastAPI used?
3. What is Uvicorn?
4. What is ASGI?
5. What is an API endpoint?
6. What are HTTP methods?
7. Difference between PUT and PATCH?
8. Difference between Path and Query Parameters?
9. What is Pydantic?
10. What is a request body?
11. What is a response model?

12. What are HTTP status codes?
13. What is CRUD?
14. What is ORM?
15. What is SQLAlchemy?
16. What is a database session?
17. Difference between primary key and foreign key?
18. What is Dependency Injection?
19. What is APIRouter?
20. What is JWT?
21. Difference between Authentication and Authorization?
22. What is OAuth2?
23. What is Middleware?
24. What is CORS?
25. Difference between synchronous and asynchronous programming?
26. What is BackgroundTasks?
27. What is WebSocket?
28. What is Alembic?
29. What is API versioning?
30. How does FastAPI communicate with React/Next.js?
31. How do you deploy a FastAPI application?
32. How do you integrate an ML model with FastAPI?

PART 20 – Practical Learning Roadmap

Meri recommendation hai ke sirf notes read na karo. Har concept ka code bhi implement karo.

10-Day FastAPI Learning Plan

Track your progress as you complete each day.

Your progress

0/10

Day 1: Introduction & Setup

FastAPI, API, installation, Uvicorn, Swagger

Day 2: API Fundamentals

Endpoints, GET, POST, PUT, PATCH, DELETE

Day 3: Parameters & Validation

Path, Query, Request Body, Pydantic

Day 4: CRUD Operations

Create a complete in-memory CRUD API

Day 5: Database Integration

SQLAlchemy, PostgreSQL, models, sessions

Day 6: Project Architecture

APIRouter, schemas, dependencies, services

Day 7: Authentication

Password hashing, JWT, OAuth2, roles

Day 8: Advanced Concepts

Middleware, CORS, async, background tasks, WebSockets

Day 9: Testing & Deployment

Pytest, TestClient, environment variables, deployment

Day 10: Final Project

Build a complete Student Management API

Reset progress

PART 21 – Final Practical Project

Recommended Project

Student Management System using FastAPI

Build a backend with these modules:

Module	Features

Authentication	Register, login, logout
Students	Add, view, update, delete
Teachers	Teacher management
Courses	Course management
Notices	Create and retrieve notices
Complaints	Submit and track complaints
Admin	Role-based access

Recommended technology stack:

- Backend: FastAPI
- Language: Python
- Database: PostgreSQL
- ORM: SQLAlchemy
- Validation: Pydantic
- Authentication: JWT
- API Testing: Swagger UI
- Frontend: Next.js
- Deployment: Railway

Ye project tumhein practically backend development ki bohat important skills sikha dega.

PART 22 – Most Important Things to Remember

Agar tum FastAPI ke sirf core concepts revise karna chahti ho, to ye sequence follow karo:

1. FastAPI fundamentals
2. HTTP methods
3. Path and Query Parameters
4. Request Body
5. Pydantic
6. Response Models
7. HTTPException
8. CRUD
9. SQLAlchemy
10. Database Relationships
11. Dependency Injection
12. APIRouter
13. JWT Authentication
14. Middleware and CORS
15. Async/Await
16. Testing and Deployment

My advice: Pehle Day 1–5 strong karo. Jab CRUD aur database integration samajh aa jaye, tab authentication aur advanced concepts par move karna.

Useful Official Resources

- FastAPI Official Documentation
- FastAPI Tutorial
- Pydantic Documentation
- SQLAlchemy Documentation

Next step: Tumhare liye best approach ye hogi ke hum FastAPI ko practical coding ke through Day 1 se start karein, har concept ka code likhein, Swagger par test karein aur phir gradually complete backend build karein.